

第 20 章

K 近邻算法

机器学习算法是从数据中产生模型，也就是进行学习的算法（下文也简称为算法）。我们把经验提供给算法，它就能够根据经验数据产生模型。在面对新的情况时，模型就会为我们提供判断（预测）结果。例如，我们根据“个子高、腿长、体重轻”判断一个孩子是个运动员的好苗子。把这些数据量化后交给计算机，它就会据此产生模型，在面对新情况时（判断另一个孩子能不能成为运动员），模型就会给出相应的判断。

比如，要对一组孩子进行测试，首先就要获取这组孩子的基本数据。这组数据包含身高、腿长、体重等数据，这些反映对象（也可以是事件）在某个方面的表现或者性质的事项，被称为属性或特征。而具体的值，如反映身高的“188 cm”就是特征值或属性值。这组数据的集合“（身高=188 cm，腿长=56 cm，体重=46 kg），……，（身高=189 cm，腿长=55 cm，体重=48 kg）”，称为数据集，其中每个孩子的数据称为一个样本。

从数据中学得模型的过程称为学习（learning）或者训练（training）。在训练过程中所使用的数据称为训练数据，其中的每个样本称为训练样本，训练样本所组成的集合称为训练集。

当然，如果希望获取一个模型，除了有数据，还需要给样本贴上对应的标签（label）。例如，“（（个子高、腿长、体重轻），好苗子）”。这里的“好苗子”就是标签，通常我们将拥有了标签的样本称为“样例”。

学得模型后，为了测试模型的效果，还要对其进行测试，被测试的样本称为测试样本。输入测试样本时，并不提供测试样本的标签（目标类别），而是由模型决定样本的标签（属于哪个类别）。比较测试样本预测的标签与实际样本标签之间的差别，就可以计算出模型的精确度。

大多数的机器学习算法都来源于日常生活实践。K 近邻算法是最简单的机器学习算法之一，主要用于将对象划分到已知类中，在生活中被广泛使用。例如，教练要选拔一批长跑运动员，如何选拔呢？他使用的可能就是 K 近邻算法，会选择个子高、腿长、体重轻，膝、踝关节围度小，跟腱明显，足弓较大者作为候选人。他会觉得这样的孩子有运动员的潜质，或者说这些孩子的特征和运动员的特征很接近。

本章从理论基础、手写数字识别算法、手写数字识别实例等角度介绍 K 近邻算法。

20.1 理论基础

K 近邻算法的本质是将指定对象根据已知特征值分类。例如，看到一对父子，一般情况下，通过判断他们的年龄，能够马上分辨出哪位是父亲，哪位是儿子。这是通过年龄属性的特征值



来划分的。

上述例子是最简单的根据单个特征维度做的分类，在实际场景中，情况可能更复杂，有多个特征维度。例如，为一段运动视频分类，判断这段视频是乒乓球比赛还是足球比赛。

为了确定分类，需要定义特征。这里定义两个特征，一个是运动员“挥手”的动作，另一个是运动员“踢脚”的动作。当然，我们不能一看到“挥手”动作就将视频归类为“乒乓球比赛”，因为我们知道某些足球运动员习惯在运动场上通过挥手来跟队友进行交流。同样，我们也不能一看到“踢脚”动作就将视频归类为“足球比赛”，因为有些乒乓球运动员会通过“踢脚”动作来表达自己的感情。

我们分别统计在某段特定时间内，视频中“挥手”和“踢脚”动作的次数，发现如下规律：

- 在乒乓球比赛的视频中，“挥手”的次数远多于“踢脚”的次数。
- 在足球比赛的视频中，“踢脚”的次数远多于“挥手”的次数。

根据对一组视频的分析，得到如表 20-1 所示的数据。

表 20-1 统计表格

视频编号	视频类型	挥手次数	踢脚次数
A	乒乓球比赛	4801	164
B	乒乓球比赛	4603	308
C	足球比赛	120	3866
D	乒乓球比赛	4417	412
E	乒乓球比赛	3367	526
F	乒乓球比赛	4335	140
G	乒乓球比赛	4222	365
H	乒乓球比赛	3427	190
I	足球比赛	130	4603
J	足球比赛	177	3332
K	乒乓球比赛	4980	532
L	乒乓球比赛	4240	258
M	乒乓球比赛	3040	556
N	乒乓球比赛	3521	300
O	乒乓球比赛	4763	256
P	足球比赛	259	4811
Q	足球比赛	369	4412
R	足球比赛	129	4143
S	乒乓球比赛	3863	236
T	足球比赛	365	4661
U	足球比赛	104	3130
V	足球比赛	172	4704
W	乒乓球比赛	4413	301
X	足球比赛	106	3596
Y	乒乓球比赛	3096	129
Z	足球比赛	589	4816

为了方便观察，将上述数据绘制为散点图，如图 20-1 所示。



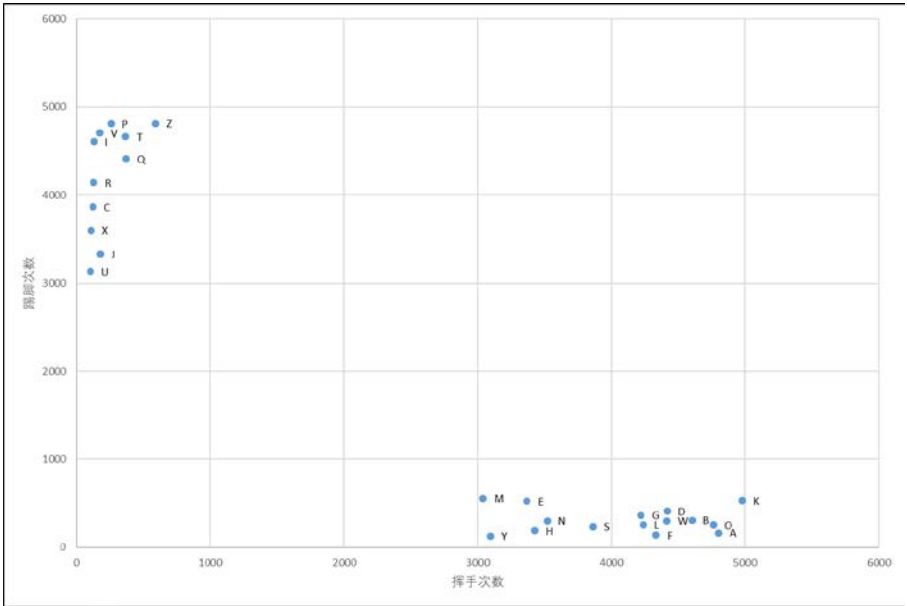


图 20-1 视频数据散点图

从图 20-1 中可以看到，数据点呈现聚集特征：

- 乒乓球比赛视频中的数据点聚集在 x 轴坐标为 $[3000, 5000]$ ， y 轴坐标为 $[1, 500]$ 的区域。
- 足球比赛视频中的数据点聚集在 y 轴坐标为 $[3000, 5000]$ ， x 轴坐标为 $[1, 500]$ 的区域。

此时，有一个视频 **Test**，经过统计得知其中出现 2000 次“挥手”动作，100 次“踢脚”动作。如果在图 20-1 中标注其位置，可以发现视频 **Test** 的位置最近的邻居是乒乓球比赛视频，因此可判断该视频是乒乓球比赛视频。

上面的例子是一个比较极端的例子，非黑即白，而实际的分类数据中往往参数非常多，判断起来也不会如此简单。因此，为了提高算法的可靠性，在实施时会取 k 个近邻点，这 k 个点中属于哪一类的较多，然后将当前待识别点划分为哪一类。为了方便判断， k 值通常取奇数，这和为了能得到明确的投票结果通常将董事会成员安排为奇数的道理是一样的。

例如，已知某知名双胞胎艺人 **A** 和 **B** 长得很像，如果要判断一张图像 **T** 上的人物到底是艺人 **A** 还是艺人 **B**，则采用 K 近邻算法实现的具体步骤如下：

(1) 收集艺人 **A** 和艺人 **B** 的照片各 100 张。

(2) 确定几个用来识别人物的重要特征，并使用这些特征来标注艺人 **A** 和 **B** 的照片。例如，根据某 4 个特征，每张照片可以表示为 $[156, 34, 890, 457]$ 这样的形式（即一个样本点）。按照上述方式，获得艺人 **A** 的 100 张照片的数据集 **FA**，艺人 **B** 的 100 张照片的数据集 **FB**。此时数据集 **FA**、**FB** 中的元素都是上述特征值的形式，每个集合中各有 100 个这样的特征值。简而言之，就是使用数值来表示照片，得到艺人 **A** 的数值特征集（数据集）**FA**、艺人 **B** 的数值特征集 **FB**。

(3) 计算待识别图像 **T** 的特征，并使用特征值表示图像 **T**。例如，图像 **T** 的特征值 **TF** 可能为 $[257, 896, 236, 639]$ 。

(4) 计算图像 **T** 的特征值 **TF** 与 **FA**、**FB** 中各特征值之间的距离。

(5) 找出产生其中 k 个最短距离的样本点 (找出离 T 最近的 k 个邻居), 统计 k 个样本点中属于 FA 和 FB 的样本点个数, 属于哪个数据集的样本点多, 就将 T 确定为哪个艺人的图像。例如, 找到 11 个最近的点, 在这 11 个点中, 属于 FA 的样本点有 7 个, 属于 FB 的样本点有 4 个, 那么就确定这张图像 T 上的艺人为 A ; 反之, 如果这 11 个点中, 有 6 个样本点属于 FB , 有 5 个样本点属于 FA , 那么就确定这张图像 T 上的艺人为 B 。

以上所述就是 K 近邻算法的基本思想。

20.2 计算

计算机的“感觉”是通过逻辑计算和数值计算来实现的。所以, 在大多数的情况下, 我们要对计算机要处理的对象进行数值化处理, 将其量化为具体的值, 以便后续处理。比较典型的方法是取某几个固定的特征, 然后将这些特征量化。例如, 在人脸识别的过程中, 可以根据人脸部器官的形状描述以及它们之间的距离特性来获取有助于分类的特征数据。这些特征数据可能包括特征点间的距离、曲率和角度等。这样, 人脸图像就可以表示为类似于 $[156, 34, 890, 457]$ 的数据形式了。

K 近邻算法在获取各个样本的特征值之后, 计算待识别样本的特征值与各个已知分类的样本特征值之间的距离, 然后找出 k 个最邻近的样本, 根据 k 个最邻近样本中占比最高的样本所属分类, 来确定待识别样本的分类。

20.2.1 归一化

对于简单的情况, 直接计算与特征值的距离 (差距) 即可。例如, 在某影视剧中, 已经通过技术手段获知犯罪嫌疑人身高为 186 cm, 受害人身高为 172 cm。而面对警察, 甲、乙二人都宣称自己是受害人。此时, 我们可以通过测量二人身高判定谁是真正的受害人:

- 甲身高为 185 cm, 与嫌疑人身高的距离 $= 186 - 185 = 1\text{cm}$, 与受害人身高的距离 $= 185 - 172 = 7\text{cm}$ 。甲的身高与嫌疑人更接近, 因此确认甲为嫌疑人。
- 乙身高为 173 cm, 与嫌疑人身高的距离 $= 186 - 173 = 13\text{cm}$, 与受害人身高的距离 $= 173 - 172 = 1\text{cm}$ 。乙的身高与受害人更接近, 因此确认乙为受害人。

上面的例子是非常简单的特例。而在实际场景中, 可能需要通过更多参数进行判断。例如, 在一部国外影视剧中, 警察通过技术手段获知嫌疑人的身高为 180 cm, 缺一根手指; 受害人身高为 173cm, 十指健全。此时, 前来投案的甲、乙二人都宣称自己是受害人。

当有多个参数时, 一般将这些参数构成列表 (数组) 进行综合判断。本例以 (身高, 手指数量) 作为特征。因此, 嫌疑人的特征值为 $(180, 9)$, 受害人的特征值为 $(173, 10)$ 。

此时, 可以对二人进行以下判断:

- 甲身高为 175 cm, 缺一根手指, 甲的特征值为 $(175, 9)$ 。
 - 甲与嫌疑人特征值的距离 $= (180 - 175) + (9 - 9) = 5$
 - 甲与受害人特征值的距离 $= (175 - 173) + (10 - 9) = 3$

此时, 甲的特征值与受害人更接近, 断定甲为受害人。



- 乙身高为 178 cm，十指健全，乙的特征值为(178, 10)。
- 乙与嫌疑人特征值的距离 = $(180-178) + (10-9) = 3$
- 乙与受害人特征值的距离 = $(178-173) + (10-10) = 5$

此时，乙与嫌疑人的特征值更接近，断定乙为嫌疑人。

当然，我们知道上述结果是错误的。因为身高、手指数量有着不同的量纲（权值），所以在计算与特征值的距离时要充分考虑不同参数之间的权值。通常情况下，由于各个参数的量纲不一致等原因，需要对参数进行处理，让所有参数具有相等的权值。

一般情况下，对参数进行归一化处理即可。做归一化时，通常使用特征值除以所有特征值中的最大值（或者最大值与最小值的差）。例如，上例中用身高除以最高身高 180（cm），手指数量除以 10（10 根手指），得到新的特征值，计算方式为：

$$\text{归一化特征} = (\text{身高}/\text{最高身高 } 180, \text{手指数量}/10)$$

因此，经过归一化以后：

- 嫌疑人的特征值为 $(180/180, 9/10) = (1, 0.9)$
- 受害人的特征值为 $(173/180, 10/10) = (0.96, 1)$

此时，可以根据归一化以后的特征值，对二人进行判断：

- 甲的特征值为 $(175/180, 9/10) = (0.97, 0.9)$
 - 甲与嫌疑人特征值的距离 = $(1-0.97) + (0.9-0.9) = 0.03$
 - 甲与受害人特征值的距离 = $(0.97-0.96) + (1-0.9) = 0.11$

此时，甲与犯罪嫌疑人的特征值更接近，断定甲为犯罪嫌疑人。

- 乙的特征值为 $(178/180, 10/10) = (0.99, 1)$
 - 乙与嫌疑人的特征值距离 = $(1-0.99) + (1-0.9) = 0.11$
 - 乙与受害人的特征值距离 = $(0.99-0.96) + (1-1) = 0.03$

此时，乙与受害人的特征值更接近，断定乙为受害人。

20.2.2 距离计算

在前面的讨论中，我们多次计算了距离。使用的方式是先将特征值中对应的元素相减，然后再求和。例如，有(身高, 体重)形式的特征值 A(185, 75)和 B(175, 86)，下面判断 C(170, 80)与特征值 A 和特征值 B 的距离：

- C 与 A 的距离 = $(185-170) + (75-80) = 15+(-5) = 10$
- C 与 B 的距离 = $(175-170) + (86-80) = 5+6 = 11$

通过计算，C 与 A 的距离更近，所以将 C 划归为 A 所属的分类。

当然，我们知道上述判断是错误的，因为在计算 C 与 A 的距离时存在负数，它抵消了一部分正数。所以，为了避免这种正负相抵的情况，我们通常会计算绝对值的和：

- C 与 A 的距离 = $|185-170| + |75-80| = 15+5 = 20$
- C 与 B 的距离 = $|175-170| + |86-80| = 5+6 = 11$



取绝对值后再求和，计算出 C 与 B 的距离更近，将 C 归为 B 所属的分类。这种用绝对值之和表示的距离，称为曼哈顿距离。

像这样计算距离已经基本满足要求了，但是还有更好的方法。例如，可以引入计算平方和的方式。此时的计算方法是：

- C 与 A 的距离 = $(185-170)^2 + (75-80)^2$
- C 与 B 的距离 = $(175-170)^2 + (86-80)^2$

更普遍的形式是计算平方和的平方根，这种距离就是被广泛使用的欧氏距离，它的计算方法是：

- C 与 A 的距离 = $\sqrt{(185-170)^2 + (75-80)^2}$
- C 与 B 的距离 = $\sqrt{(175-170)^2 + (86-80)^2}$

20.2 手写数字识别的原理

20.1 节我们仅仅取了两个特征维度进行说明。在实际应用中，可能存在着更多特征维度需要计算。下面以手写数字识别为例进行简单的介绍。

假设我们要让程序识别图 20-2 中上方的数字（当然，你一眼就知道是“8”，但是现在要让计算机识别出来）。识别的方式是，依次计算该数字图像（即写有数字的图像）与下方数字图像的距离，与哪个数字图像的距离最近（此时 $k=1$ ），就认为它与哪幅图像最像，从而确定这幅图像中的数字是多少。

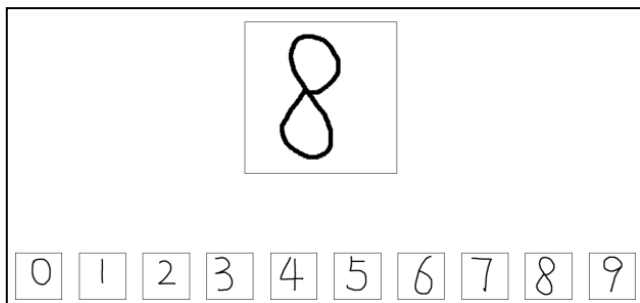


图 20-2 手写数字识别示例

下面分别从特征值提取和数字识别两方面展开介绍。

1. 特征值提取

步骤 1：我们把数字图像划分成很多小块，如图 20-3 所示。该图中每个数字被分成 5 行 4 列，共计 $5 \times 4 = 20$ 个小块。此时，每个小块是由很多个像素点构成的。当然，也可以将每一个像素点理解为一个更小的子块。

为了叙述上的方便，将这些小块表示为 B (Bigger)，将 B 内的像素点，记为 S (Smaller)。因此，待识别的数字“8”的图像可以理解为：



- 由 5 行 4 列，共计 $5 \times 4 = 20$ 个小块 B 构成。
- 每个小块 B 内其实是由 $M \times N$ 个像素（更小块 S）构成的。为了描述上的方便，假设每个小块大小为 $10 \times 10 = 100$ 个像素。

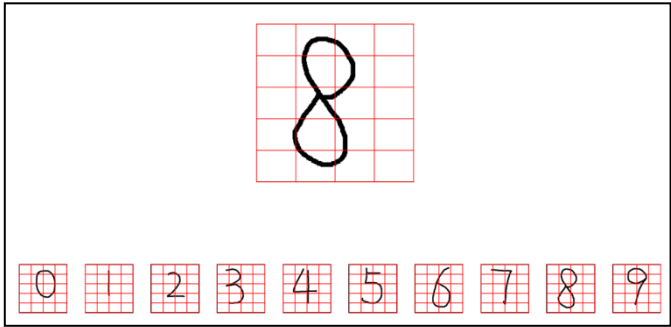


图 20-3 划分子块

步骤 2：计算每个小块 B 内，有多少个黑色的像素点。或者说，计算每个小块 B 内有多少个更小块 S 是黑色的。

仍以数字“8”的图像为例，其第 1 行中：

- 第 1 个小块 B 共有 0 个像素点（更小块 S）是黑色的，记为 0。
- 第 2 个小块 B 共有 28 个像素点（更小块 S）是黑色的，记为 28。
- 第 3 个小块 B 共有 10 个像素点（更小块 S）是黑色的，记为 10。
- 第 4 个小块 B 共有 0 个像素点（更小块 S）是黑色的，记为 0。

以此类推，计算出数字“8”的图像中每一个小块 B 中有多少个像素点是黑色的，如图 20-4 所示。我们观察后会发现，不同的数字图像中每个小块 B 内黑色像素点的数量是不一样的。正是这种不同，使我们能用该数量（每个小块 B 内黑色像素点的个数）作为特征来表示每一个数字。

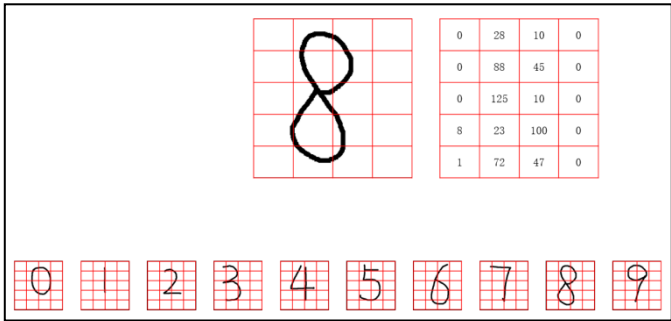


图 20-4 每个小块 B 内黑色像素点的个数

步骤 3：有时，为了处理上的方便，我们会把得到的特征值排成一行（写为数组形式），如图 20-5 所示。

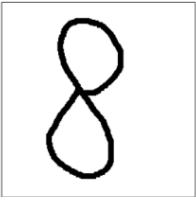
0	28	10	0
0	88	45	0
0	125	10	0
8	23	100	0
1	72	47	0

[0,28,10,0,0,88,0,45,0,125,10,0,8,23,100,0,1,72,47,0]

图 20-5 将特征值排成一行

当然，在 Python 里完全没有必要这样做，因为 Python 可以非常方便地直接处理图 20-5 中上方数组（array）形式的数据。这里为了说明上的方便，仍将其特征值处理为一行数字的形式。

经过上述处理，数字“8”图像的特征值变为一行数字，如图 20-6 所示。



[0,28,10,0,0,88,0,45,0,125,10,0,8,23,100,0,1,72,47,0]

图 20-6 数字“8”图像的特征值

步骤 4：与数字“8”的图像类似，每个数字图像的特征值都可以用一行数字来表示。从某种意义上来说，这一行数字类似于我们的身份证号码，一般来说，具有唯一性。

按照同样的方式，获取每个数字图像的特征值，如图 20-7 所示。

0	[0, 45,0,125,10,0,8,23,100,0,1, 23,11,0,0,88,0,72,47,0]
2	[0, 4,0,12,10,0,8,23,26,10,0,0,88,0, 100,0,1,72,47,0]
3	[0, 23,100,0,1,72,47,0,28,10,0,0,88,0,45,0,125,10,0,8]
4	[0,28,10,0,0,88,0,45,0,125,10,0,8,23,100,0,1,72,47,5]
5	[0,12,140,50,7,8,0,8,23,10,0,1,,45,0,125,10,0,72,6,3]
6	[0,28,10,0,0,7,0,45,0,25,10,0,8,23,100,0,1,72,5,9]
7	[0,128,10,0,0,8,0,45,0,15,10,0,8,23,100,0,1,72,1,7]
8	[70,8,10,0,0,88,0,45,0,125,10,0,8,23,100,0,1,2,4,3]
1	[0,218,10,0,11,8,80,45,0,12,10,0,8,23,10,0,1,7,7,4]
9	[0,2,10,0,0,88,0,45,0,25,10,0,8,23,100,0,1,72,47,0]

图 20-7 每个数字图像的特征值

2. 数字识别

数字识别要做的就是比较待识别图像与图像集中的哪个图像最近。这里，最近指的是二者之间的欧氏距离最短。



本例中为了便于说明和理解进行了简化，将原来下方的 10 个数字减少为 2 个（也即将分类从 10 个减少为 2 个）。假设要识别的图像为图 20-8 中上方的数字“8”图像，需要判断该图像到底属于图 20-8 中下方的数字“8”图像的分类还是数字“7”图像的分类。

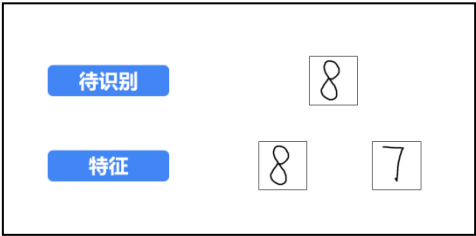


图 20-8 待识别图像与特征图像

步骤 1：提取特征值，分别提取待识别图像的特征值和特征图像的特征值。
为了说明和理解上的方便，将特征进行简化，每个数字图像只提取 4 个特征值（划分为 $2 \times 2 = 4$ 个子块 B），如图 20-9 所示。此时，提取到的特征值分别为：

- 待识别的数字“8”图像：[3, 7, 8, 13]
- 数字“8”特征图像：[3, 6, 9, 12]
- 数字“7”特征图像：[8, 1, 2, 98]

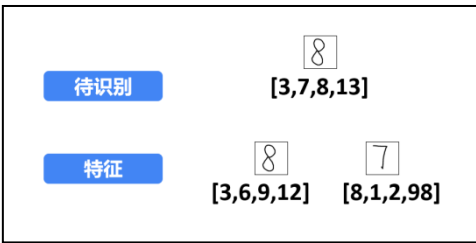


图 20-9 重新计算的特征值

步骤 2：计算距离。按照 20.1 节介绍的欧氏距离计算方法，计算待识别图像与特征图像之间的距离。

首先，计算待识别的数字“8”图像与下方的数字“8”特征图像之间的距离，如图 20-10 所示。计算二者之间的距离：

$$\text{距离 } 1 = \sqrt{(3 - 3)^2 + (7 - 6)^2 + (8 - 9)^2 + (13 - 12)^2} = \sqrt{3}$$

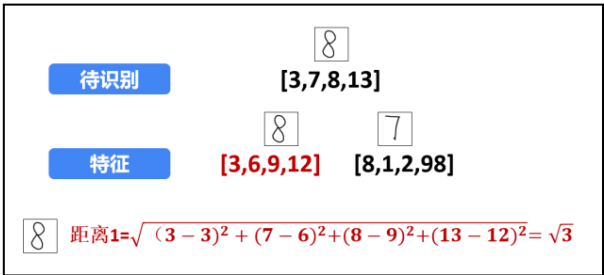


图 20-10 计算距离 1

接下来, 计算待识别的数字“8”图像与数字“7”特征图像之间的距离, 如图 20-11 所示。二者之间的距离为:

$$\text{距离 } 2 = \sqrt{(3-8)^2 + (7-1)^2 + (8-2)^2 + (13-98)^2} = \sqrt{7322}$$

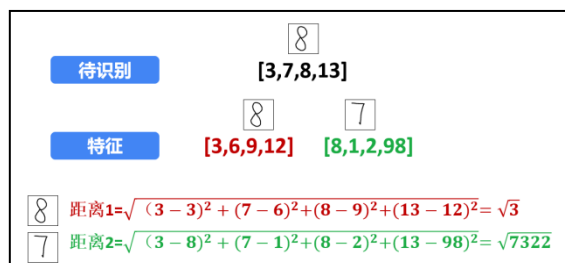


图 20-11 计算距离 2

通过计算可知, 待识别的数字“8”图像:

- 与数字“8”特征图像的距离为 $\sqrt{3}$ 。
- 与数字“7”特征图像的距离为 $\sqrt{7322}$ 。

步骤 3: 识别。

根据计算的距离, 待识别的数字“8”图像与数字“8”特征图像的距离更近。所以, 将待识别的数字“8”图像识别为数字“8”特征图像所代表的数字“8”。

上面介绍的是 K 近邻算法只考虑最近的一个邻居的情况, 相当于 K 近邻中 $k=1$ 的情况。在实际操作中, 为了提高可靠性, 需要选用大量的特征值。例如, 每个数字都选用不同的形态的手写体 100 个, 对于 0~9 这 10 个数字, 共需要 $100 \times 10 = 1000$ 幅特征图像。在识别数字时, 分别计算待识别的数字图像与这些特征图像之间的距离。这时, 可以将 k 调整为稍大的值, 例如 $k=11$, 然后看看其最近的 11 个邻居分属于哪些特征图像。例如, 其中:

- 有 8 个属于数字“6”特征图像。
- 有 2 个属于数字“8”特征图像。
- 有 1 个属于数字“9”特征图像。

通过判断, 当前待识别的数字为数字“6”特征图像所代表的数字“6”。

20.3 自定义函数手写数字识别

OpenCV 提供了函数 `cv2.KNearest()` 用来实现 K 近邻算法, 在 OpenCV 中可以直接调用该函数。为了进一步了解 K 近邻算法及其实现方式, 本节首先使用 Python 和 OpenCV 实现一个识别手写数字的实例。

【例 20.1】 编写程序, 演示 K 近邻算法。

在本例中, 0~9 的每个数字都有 10 个特征值。例如, 数字“0”的特征值如图 20-12 所示。为了便于描述, 将所有这些用于判断分类的图像称为特征图像。



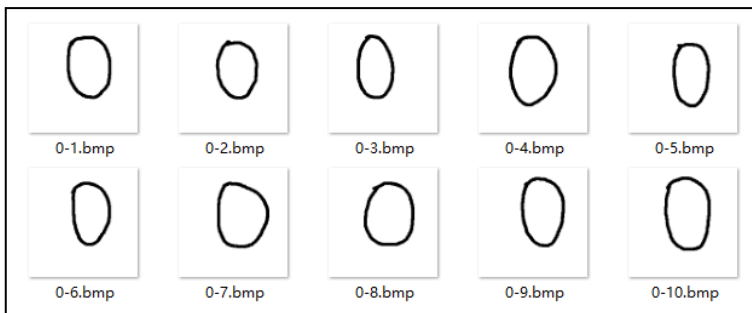


图 20-12 数字“0”的特征图像

下面分步骤实现手写数字的识别。

1. 数据初始化

对程序中要用到的数据进行初始化。涉及的数据主要有路径信息、图像大小、特征值数量、用来存储所有特征值的数据等。

本例中：

- 特征图像存储在当前路径的“image”文件夹下。
- 用于判断分类的特征值有 100 个（对应 100 幅特征图像）。
- 特征图像的行数（高度）、列数（宽度）可以通过程序读取。也可以在图像上单击鼠标右键后通过查找属性值来获取。这里采用设置好的特征图像集，每个特征图像都是高 240 行、宽 240 列。

根据上述已知条件，对要用到的数据初始化：

```
s='image\\' # 图像所在的路径
num=100    # 共有特征值的数量
row=240    # 特征图像的行数
col=240    # 特征图像的列数
a=np.zeros((num,row,col)) # a 用来存储所有特征的值
```

2. 读取特征图像

本步骤将所有特征图像读入到 a 中。共有 10 个数字，每个数字有 10 个特征图像，采用嵌套循环语句完成读取。具体代码如下：

```
n=0 # n 用来存储当前图像的编号。
for i in range(0,10):
    for j in range(1,11):
        a[n,:,:]=cv2.imread(s+str(i)+'\\'+str(i)+'-'+str(j)+'.bmp',0)
        n=n+1
```

3. 提取特征图像的特征值

在提取特征值时，可以计算每个子块内黑色像素点的个数，也可以计算每个子块内白色像素点的个数。这里我们选择计算白色像素点（像素值为 255）的个数。按照上述思路，图像映射到特征值的关系如图 20-13 所示。

表 20-2 数组 d 的索引与特征图像的对应关系

索引	关系
0~9	对应着待识别图像到数字“0”的 10 个特征图像的距离
10~19	对应着待识别图像到数字“1”的 10 个特征图像的距离
20~29	对应着待识别图像到数字“2”的 10 个特征图像的距离
30~39	对应着待识别图像到数字“3”的 10 个特征图像的距离
40~49	对应着待识别图像到数字“4”的 10 个特征图像的距离
50~59	对应着待识别图像到数字“5”的 10 个特征图像的距离
60~69	对应着待识别图像到数字“6”的 10 个特征图像的距离
70~79	对应着待识别图像到数字“7”的 10 个特征图像的距离
80~89	对应着待识别图像到数字“8”的 10 个特征图像的距离
90~99	对应着待识别图像到数字“9”的 10 个特征图像的距离

如果将索引号整除 10，得到的值正好是其对应的特征图像上的数字。例如 `d[34]` 对应着待识别图像到数字“3”的第 4 个特征图像的欧式距离。而将 34 整除 10，得到 `int(34/10) = 3`，正好是其对应的特征图像上的数字。

确定了索引与特征图像的关系，下一步可以通过计算索引达到数字识别的目的。

6. 获取k个最短距离及其索引

从计算得到的所有距离中，选取 *k* 个最短距离，并计算出这 *k* 个最短距离对应的索引。具体实现方式是：

- 每次找出最短的距离（最小值）及其索引（下标），然后将该最小值替换为最大值。
- 重复上述过程 *k* 次，得到 *k* 个最短距离对应的索引。

每次将最小值替换为最大值，是为了确保该最小值在下一次查找最小值的过程中不会再次被找到。例如，要在数字序列“11, 6, 3, 9”内依次找到从小到大的值。

- 第 1 次找到了最小值“3”，同时将“3”替换为“11”。此时，要查找的序列变为“11, 6, 11, 9”。
- 第 2 次查找最小值时，在序列“11, 6, 11, 9”内找到的最小值是数字“6”，同时将“6”替换为最大值“11”，得到序列“11,11,11,9”。

不断地重复上述过程，依次在第 3 次找到最小值“9”，在第 4 次找到最小值“11”。当然，在本例中查找的是数值，具体实现时查找的是索引值。

根据上述思路，编写代码如下：

```
d=d.tolist()
temp=[]
Inf = max(d)
#print(Inf)
k=7
for i in range(k):
    temp.append(d.index(min(d)))
    d[d.index(min(d))]=Inf
```

7. 识别

根据计算出来的 k 个最小值的索引, 结合表 20-2 就可以确定索引所对应的数字。具体实现方法是将索引值整除 10, 得到对应的数字。

例如, 在 $k=11$ 时, 得到最小的 11 个值所对应的索引依次为: 66、60、65、63、68、69、67、78、89、96、32。它们所对应的特征图像如表 20-3 所示。

表 20-3 索引对应的特征图像

索引	对应关系
66	数字“6”的第 6 个特征图像
60	数字“6”的第 0 个特征图像
65	数字“6”的第 5 个特征图像
63	数字“6”的第 3 个特征图像
68	数字“6”的第 8 个特征图像
69	数字“6”的第 9 个特征图像
67	数字“6”的第 7 个特征图像
78	数字“7”的第 8 个特征图像
89	数字“8”的第 9 个特征图像
96	数字“9”的第 6 个特征图像
32	数字“3”的第 2 个特征图像

这说明, 当前待识别图像与数字“6”的第 6 个特征图像距离最近; 接下来, 距离最近的第 2 个特征图像是数字“6”的第 0 个特征图像 (序号从 0 开始); 以此类推, 距离最近的第 11 个特征图像是数字“3”的第 2 个特征图像。

上述结果说明, 与待识别图像距离最近的特征图像中, 有 7 个是数字“6”的特征图像。所以, 待识别图像是数字“6”。

下面讨论如何通过程序识别数字。已知将索引整除 10, 就能得到对应特征图像上的数字, 因此对于上述索引整除 10:

$$(66, 60, 65, 63, 68, 69, 67, 78, 89, 96, 32) \text{ 整除 } 10 = (6, 6, 6, 6, 6, 6, 7, 8, 9, 3)$$

为了叙述上的方便, 将上述整除结果标记为 dr , 在 dr 中出现次数最多的数字, 就是识别结果。对于上例, dr 中“6”的个数最多, 所以识别结果就是数字“6”。

这里我们借助索引判断一组数字中哪个数字出现的次数最多:

- 建立一个数组 r , 让其元素的初始值都是 0。
- 依次从 dr 中取数字 n , 将数组 r 索引位置为 n 的值加 1。例如, 从 dr 中取到的第 1 个数字为“6”, 将 $r[6]$ 加上 1; 从 dr 中取到第 2 个数字也为“6”, 将 $r[6]$ 加上 1; 以此类推, 对于 $dr=[6, 6, 6, 6, 6, 6, 7, 8, 9, 3]$, 得到数组 r 的值为 $[0, 0, 0, 1, 0, 0, 7, 1, 1, 1]$ 。

在数组 r 中:

- $r[0]=0$, 表示在 dr 中不存在值为 0 的元素。
- $r[3]=1$, 表示在 dr 中有 1 个“3”。
- $r[6]=7$, 表示在 dr 中有 7 个“6”。
- $r[7]=1$, 表示在 dr 中有 1 个“7”。



- `r[8]=1`，表示在 `dr` 中有 1 个“8”。
- `r[9]=1`，表示在 `dr` 中有 1 个“9”。
- `r` 中其余为 0 的值，表示其对应的索引在 `dr` 中不存在。

根据上述思路，编写代码如下：

```
temp=[i/10 for i in temp]
# 数组 r 用来存储结果，r[0]表示 K 近邻中“0”的个数，r[n]表示 K 近邻中“n”的个数
r=np.zeros(10)
for i in temp:
    r[int(i)]+=1
print('当前的数字可能为:'+str(np.argmax(r)))
```

上述过程是分步骤的分析结果，以下是全部源代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 读取样本（特征）图像的值
s='image\\' # 图像所在路径
num=100 # 样本总数
row=240 # 特征图像的行数
col=240 # 特征图像的列数
a=np.zeros((num,row,col)) # 存储所有样本的数值
#print(a.shape)
n=0 # 存储当前图像的编号
for i in range(0,10):
    for j in range(1,11):
        a[n, :, :] = cv2.imread(s+str(i)+'\\'+str(i)+'-'+str(j)+'.bmp', 0)
        n=n+1
#提取样本图像的特征
feature=np.zeros((num,round(row/5),round(col/5))) # 用来存储所有样本的特征值
#print(feature.shape) # 看看特征值的形状是什么样子
#print(row) # 看看 row 的值，有多少个特征值（100）

for ni in range(0,num):
    for nr in range(0,row):
        for nc in range(0,col):
            if a[ni,nr,nc]==255:
                feature[ni,int(nr/5),int(nc/5)]+=1
f=feature # 简化变量名称
#####计算当前待识别图像的特征值
o=cv2.imread('image\\test\\5.bmp',0) # 读取待识别图像
##读取图像值
of=np.zeros((round(row/5),round(col/5))) # 存储待识别图像的特征值
for nr in range(0,row):
    for nc in range(0,col):
        if o[nr,nc]==255:
            of[int(nr/5),int(nc/5)]+=1
```



```

###开始计算，识别数字，计算最邻近的若干个数字是多少，判断结果
d=np.zeros(100)
for i in range(0,100):
    d[i]=np.sum((o-f[i, :, :])*(o-f[i, :, :]))
#print(d)
d=d.tolist()
temp=[]
Inf = max(d)
#print(Inf)
k=7
for i in range(k):
    temp.append(d.index(min(d)))
    d[d.index(min(d))]=Inf
#print(temp)  #看看都被识别为哪些特征值
temp=[i/10 for i in temp]
# 也可以返回去处理为 array，使用函数处理
#temp=np.array(temp)
#temp=np.trunc(temp/10)
#print(temp)
# 数组 r 用来存储结果，r[0]表示 K 近邻中“0”的个数，r[n]表示 K 近邻中“n”的个数
r=np.zeros(10)
for i in temp:
    r[int(i)]+=1
#print(r)
print('当前的数字可能为:'+str(np.argmax(r)))

```

20.4 K 近邻模块的基本使用

在 OpenCV 中，不需要自己编写复杂的函数实现 K 近邻算法，直接调用其自带的模块函数即可。本节通过一个简单的例子介绍如何使用 OpenCV 自带的 K 近邻模块。

【例 20.2】 演示 OpenCV 自带的 K 近邻模块的使用方法。

本例中有两组位于不同位置的用于训练的数据集，如图 20-14 所示。两组数据集中，一组位于左下角；另一组位于右上角。随机生成一个数值，用 OpenCV 中的 K 近邻模块判断该随机数属于哪一个分组。

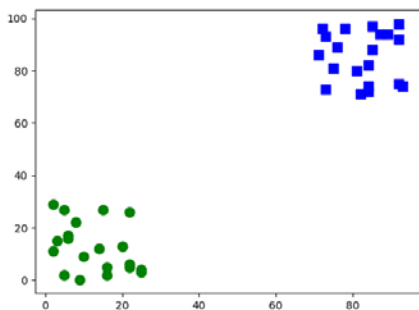


图 20-14 两组不同的数据集



上述两组数据中，位于左下角的一组数据，其 x 、 y 坐标值都在(0, 30)范围内。位于右上角的数据，其 x 、 y 坐标值都在(70, 100)范围内。

根据上述分析，创建两组数据，每组包含 20 对随机数（20 个随机数据点）：

```
rand1 = np.random.randint(0, 30, (20, 2)).astype(np.float32)
rand2 = np.random.randint(70, 100, (20, 2)).astype(np.float32)
```

- 第 1 组随机数 `rand1` 中，其 x 、 y 坐标值均位于(0, 30)区间内。
- 第 2 组随机数 `rand2` 中，其 x 、 y 坐标值均位于(70, 100)区间内。

接下来，为两组随机数分配标签：

- 将第 1 组随机数对划分为类型 0，标签为 0。
- 将第 2 组随机数对划分为类型 1，标签为 1。

然后，生成一对值在(0, 100)内的随机数对：

```
test = np.random.randint(0, 100, (1, 2)).astype(np.float32)
```

最后，使用 OpenCV 自带的 K 近邻模块判断生成的随机数对 `test` 是属于 `rand1` 所在的类型 0，还是属于 `rand2` 所在的类型 1。

根据分析，编写代码如下：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 用于训练的数据
# rand1 数据位于(0,30)
rand1 = np.random.randint(0, 30, (20, 2)).astype(np.float32)
# rand2 数据位于(70,100)
rand2 = np.random.randint(70, 100, (20, 2)).astype(np.float32)
# 将 rand1 和 rand2 拼接为训练数据
trainData = np.vstack((rand1, rand2))
# 数据标签，共两类：0 和 1
# r1Label 对应着 rand1 的标签，为类型 0
r1Label=np.zeros((20,1)).astype(np.float32)
# r2Label 对应着 rand2 的标签，为类型 1
r2Label=np.ones((20,1)).astype(np.float32)
tdLable = np.vstack((r1Label, r2Label))
# 使用绿色标注类型 0
g = trainData[tdLable.ravel() == 0]
plt.scatter(g[:,0], g[:,1], 80, 'g', 'o')
# 使用蓝色标注类型 1
b = trainData[tdLable.ravel() == 1]
plt.scatter(b[:,0], b[:,1], 80, 'b', 's')
# plt.show()
# test 为用于测试的随机数，该数在 0 到 100 之间
test = np.random.randint(0, 100, (1, 2)).astype(np.float32)
plt.scatter(test[:,0], test[:,1], 80, 'r', '*')
```

```
# 调用 OpenCV 内的 K 近邻模块, 并进行训练
knn = cv2.ml.KNearest_create()
knn.train(trainData, cv2.ml.ROW_SAMPLE, tdLable)
# 使用 K 近邻算法分类
ret, results, neighbours, dist = knn.findNearest(test, 5)
# 显示处理结果
print("当前随机数可以判定为类型: ", results)
print("距离当前点最近的 5 个邻居是: ", neighbours)
print("5 个最近邻居的距离: ", dist)
# 可以观察一下显示, 对比上述输出
plt.show()
```

运行上述程序, 显示的运行结果 (因为是随机数, 每次结果会略有不同) 为:

当前随机数可以判定为类型: `[[1.]]`

距离当前点最近的 5 个邻居是: `[[1. 1. 1. 1. 1.]]`

5 个最近邻居的距离: `[[313. 324. 338. 377. 405.]]`

同时, 程序还会显示如图 20-15 所示的运行结果。从图中可以看出, 随机点 (星号点) 距离右侧小方块 (类型为 1) 的点更近, 因此被判定为属于小方块的类型 1。

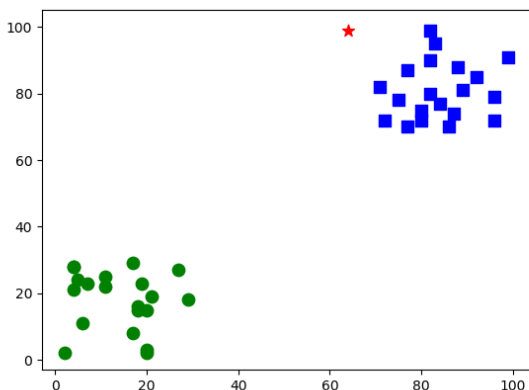


图 20-15 【例 20.2】程序的运行结果

20.5 K 近邻手写数字识别

本节使用 OpenCV 自带的 K 近邻模块识别手写数字。

【例 20.3】 修改例 20.1 中的程序, 使用 OpenCV 自带的函数完成对手写数字的识别。

根据题目的要求, 编写代码如下:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 读取样本 (特征) 图像的值
s='image\\' # 图像所在的路径
num=100 # 共有的样本数量
```



```

row=240 # 特征图像的行数
col=240 # 特征图像的列数
a=np.zeros((num,row,col)) # 用来存储所有样本的数值
#print(a.shape)
n=0 # 用来存储当前图像的编号
for i in range(0,10):
    for j in range(1,11):
        a[n,:,:]=cv2.imread(s+str(i)+'\\'+str(i)+'-'+str(j)+'.bmp',0)
        n=n+1
# 提取样本图像的特征
feature=np.zeros((num,round(row/5),round(col/5))) # 用来存储所有样本的特征值
#print(feature.shape) # 看看特征值的形状是什么样子
#print(row) # 看看 row 的值，有多少个特征值（100）
for ni in range(0,num):
    for nr in range(0,row):
        for nc in range(0,col):
            if a[ni,nr,nc]==255:
                feature[ni,int(nr/5),int(nc/5)]+=1
f=feature # 简化变量名称
# 将 feature 处理为单行形式
train = feature[:,:].reshape(-1,
                               round(row/5)*round(col/5)).astype(np.float32)
#print(train.shape)
# 贴标签，要注意，是 range(0,100)而非 range(0,101)
trainLabels = [int(i/10) for i in range(0,100)]
trainLabels=np.asarray(trainLabels)
#print(*trainLabels) # 打印测试看看标签值
##读取图像值
o=cv2.imread('image\\test\\5.bmp',0) # 读取待识别图像
of=np.zeros((round(row/5),round(col/5))) # 用来存储待识别图像的特征值
for nr in range(0,row):
    for nc in range(0,col):
        if o[nr,nc]==255:
            of[int(nr/5),int(nc/5)]+=1

test=of.reshape(-1,round(row/5)*round(col/5)).astype(np.float32)
# 调用函数识别图像
knn=cv2.ml.KNearest_create()
knn.train(train,cv2.ml.ROW_SAMPLE, trainLabels)
ret,result,neighbours,dist = knn.findNearest(test,k=5)
print("当前随机数可以判定为类型: ", result)
print("距离当前点最近的 5 个邻居是: ", neighbours)
print("5 个最近邻居的距离: ", dist)
运行上述程序，程序运行结果为：
当前随机数可以判定为类型： [[5.]]
距离当前点最近的 5 个邻居是： [[5. 3. 5. 3. 5.]]
5 个最近邻居的距离： [[77185. 78375. 79073. 79948. 82151.]]

```